

Geography +

Geospatial +

Geocoding +

Geocoding Map +

WebView Injection



Max Kleiner 5 min read · 20 hours ago

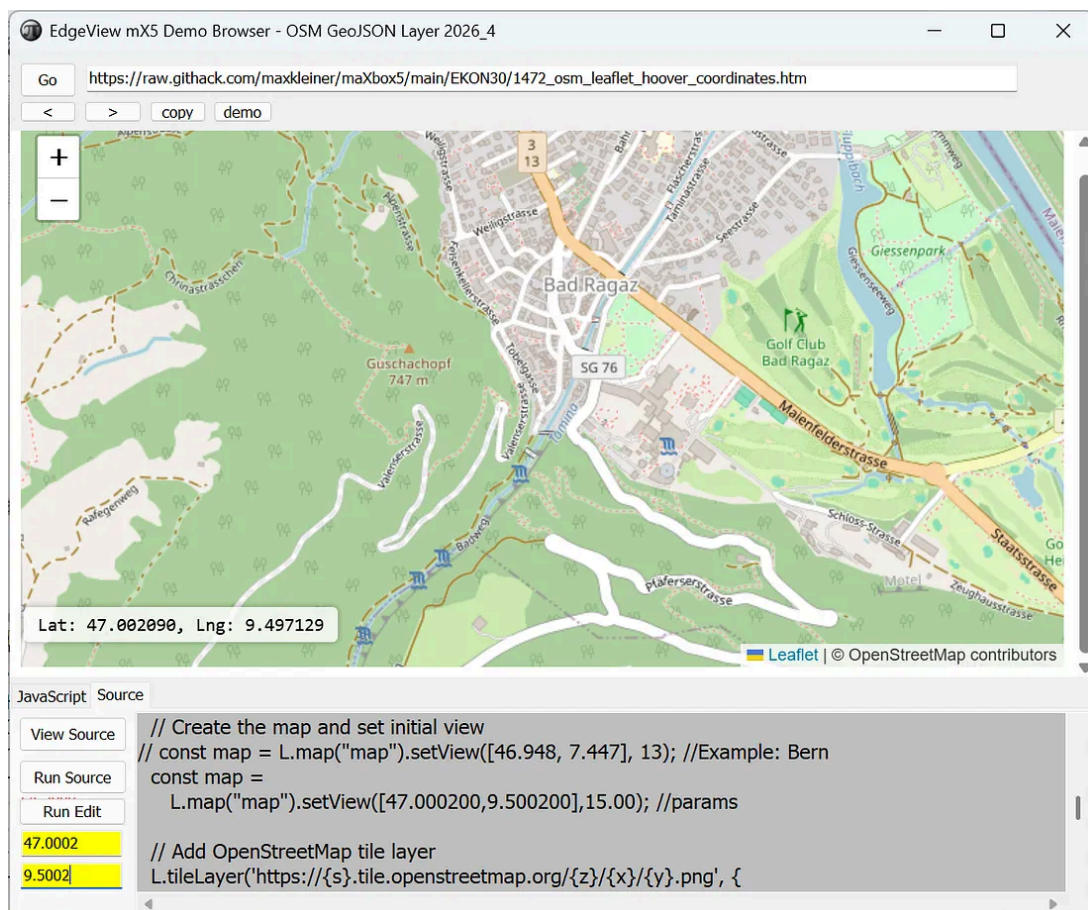


6



This type of procedure function shows the injection of a JavaScript OSM leaflet library at runtime in a rich client of the EdgeView2 component. This will be the maXbox Starter **Tutorial 192**.

This component can embed web content (HTML, CSS, and JavaScript) in your native applications with Microsoft Edge WebView2 runtime dll.



TEdgeViewForm with geo coordinates parameters to inject in JavaScript

This procedure is the `OnClick` handler for a button (`Button1`) that lives on an `TEdit` control (`Edit1`) in Delphi. Its job is to take latitude/longitude/zoom values from the UI, build/update some HTML/JavaScript content for a map view, and then refresh that view.

The button on the form above is called **Run Edit** which passes the latitude and longitude to find and set on a `OpenStreetMap`:

```
procedure TEdit1Button1Click(Sender: TObject);
    //var Image1: TImage;
    var htmlgeojs2: string;
begin
    writ('debug c: '+edt1lat.text+' '+edt2long.text)
    if SetSCRIPT_PARAMS(Format('%.6f',[strtofloat(edt1lat.text)]),
                        Format('%.6f',[strtofloat(edt2long.text)]),
                        format('%.2f',[zoomf])) then
        htmlgeojs2:= utf8encode(stringreplace((HOVER_JS_SCRIPT2),LF,CRLF,[rfReplac
edgeviewfrm.memoHTML.Text:= utf8toansi(htmlgeojs2); ////GEOJSON_Layer;
edgeviewfrm.btnSetSourceClick(self);
end;
```

Step-by-step behavior

- **Debug output**

`writ('debug_c: '+edt1lat.text+' '+edt2long.text)` writes the current text of two edit boxes (`edt1lat`, `edt2long`) to some debug log/output. This is just for troubleshooting.

- **Set script parameters**

It calls `SetSCRIPT_PARAMS` with three formatted strings:

- Latitude: `Format('%.6f',[strtofloat(edt1lat.text)])`
- Longitude: `Format('%.6f',[strtofloat(edt2long.text)])`
- Zoom: `format('%.2f',[zoomf])`

So it converts the latitude and longitude edit texts to `float`, formats them to 6 decimal places, formats the zoom to 2 decimals, and passes them into `SetSCRIPT_PARAMS`. The `if` means the following block only runs if that call returns `True`.

- **Build HTML/JS string**

If `SetSCRIPT_PARAMS` succeeds:

- It takes a constant/template string `HOVER_JS_SCRIPT2`.

- Replaces all line feeds (LF) with carriage return + line feed (CRLF) using `stringreplace(..., LF, CRLF, [rfReplaceAll])`.
- Wraps the result with `utf8encode`, assigning it to `htmlgeojs2`.
This suggests `HOVER_JS_SCRIPT2` is UTF-8 text (likely HTML/JS for a map) that needs Windows-style line endings.
- **Push HTML into a memo and refresh the view**
- `edgeviewfrm.memoHTML.Text := utf8toansi(htmlgeojs2);`
Converts the UTF-8 string to ANSI and puts it into `memoHTML` on a form called `edgeviewfrm`. That memo presumably holds the HTML source used by some embedded browser or viewer.
- `edgeviewfrm.btnSetSourceClick(self);`
Programmatically triggers the `OnClick` handler of `btnSetSource` on `edgeviewfrm`. That handler likely tells the browser control to (re)load the HTML from `memoHTML`, thus updating the displayed map with the new lat/lon/zoom data.

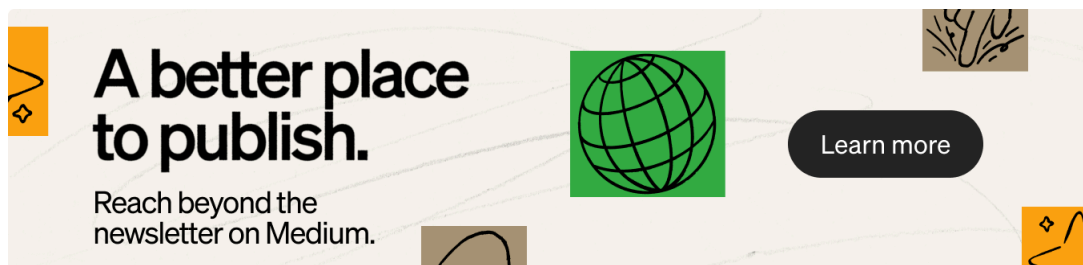
So originally this button may have been used to load an image file, but that functionality is disabled now; the procedure currently only updates the map view based on the coordinate/zoom inputs for OpenStreetMap.

The embed JScript is a function called `SetSCRIPT_PARAMS()` which injects the string html-template `HOVER_JS_SCRIPT2`. which returns the updated script at runtime:

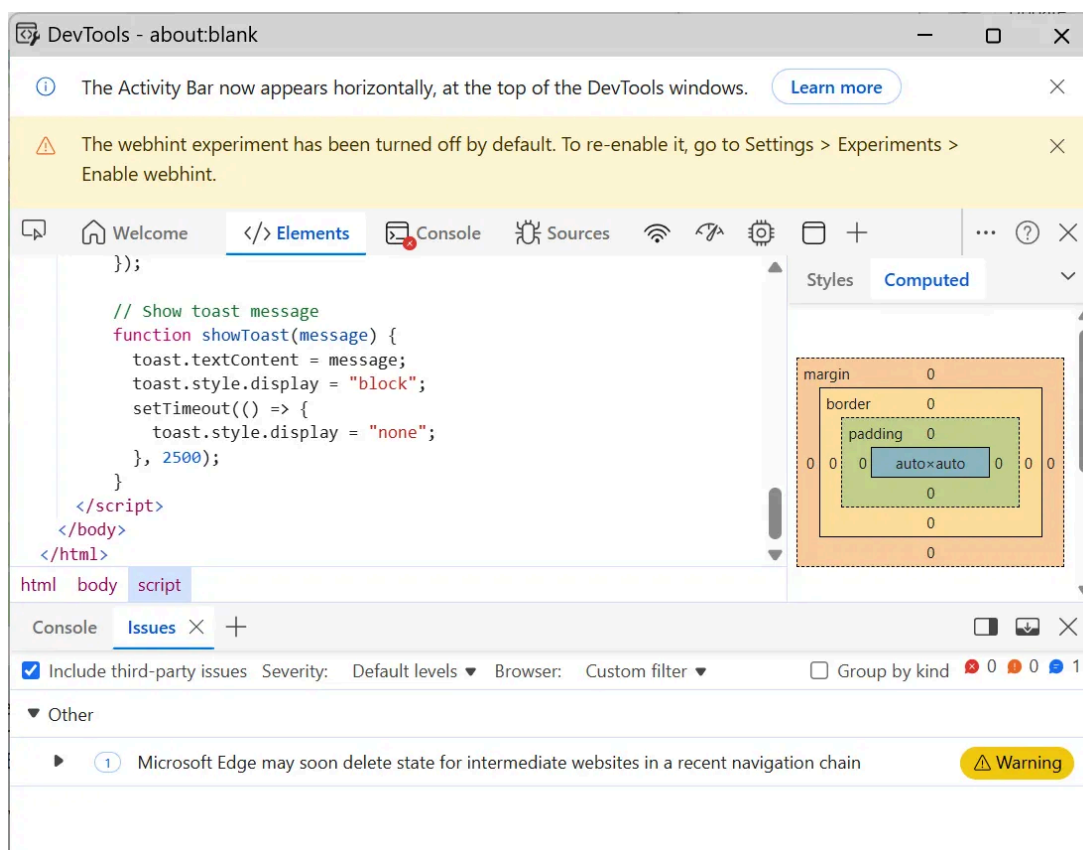
```
function SetSCRIPT_PARAMS(vlat,vlong,vzoom: string): boolean;
begin
  result:= false;
  HOVER_JS_SCRIPT2:=
    '<!DOCTYPE html>' +lf+
    '<html>' +lf+
    '<head>' +lf+
    '  <meta charset="utf-8" />' +lf+
    '  <title>Leaflet OSM Hover Coordinates on maXbox52</title>' +lf+
    '  <meta name="viewport" content="width=device-width, initial-scale=1.0">' +lf+
    '  <!-- Leaflet CSS -->' +lf+
    '  <link rel="stylesheet" href="https://unpkg.com/leaflet/dist/leaflet.css" />' +lf+
    '  <style>' +lf+
    '    #map {
    '      height: 100vh; /* Full screen height */
    '      width: 100%;
    '    }
    '  }
```

The adapted jscript will then executed with
`edgeviewfrm.btnSetSourceClick(self);`

What is the Inspect tool?



Inspect is a tool that allows you to view HTML and CSS elements on a webpage. The HTML or JavaScript code creates animation, and CSS handles web design. It can help you understand how a webpage is constructed, what font a site may be using, see what elements of a website are broken, and how long a site takes to load. You can also use Inspect to see how temporary changes to the code may look on your device.



The Inspect tool can be started with right hand click on the webview map.

Conclusion

`TEdit1Button1Click` reads lat/lon/zoom from edit controls, passes them to a script-parameter function, regenerates an HTML/JavaScript snippet for a

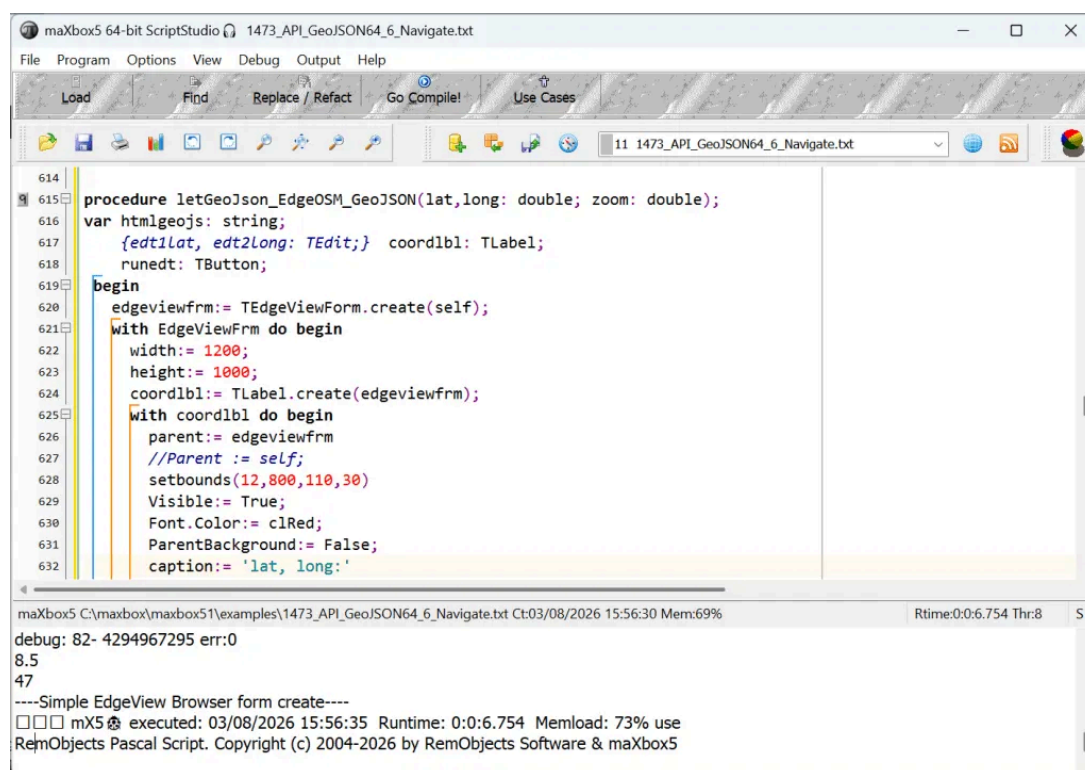
map viewer, puts it into a memo on another form, and then forces that viewer to reload the content in maXbox script engine.

The script can be found at:

[Download 1473_API_GeoJSON64_6_Navigate.txt \(maXbox5\).](#)

JavaScript injection is a technique where a user inserts their own JavaScript code into a web page. This can be done for various purposes, such as debugging, testing, or even malicious activities. In our example the script runs on a client side web form which isolates or delegates server-side scripting to a desktop rich client.

There's no evidence that WebView2 Runtime is categorically more secure than Edge. It shares the same core engine and receives similar engine-level patches, but it lacks some browser-specific protections and depends on app developers to use it safely.



https://sourceforge.net/projects/maxbox5/files/examples/1473_API_GeoJSON64_6_Navigate.txt/download

[Download 1473_API_GeoJSON64_6_Navigate.txt \(maXbox5\).](#)

To get more experiences with geo mapping and world coordinates I can recommend the following site:

GPS Coordinates, Latitude and Longitude with Interactive Maps

Leaflet Library

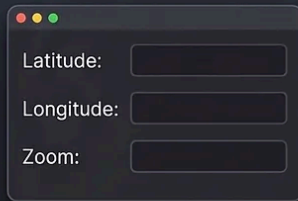
In the context of the provided source, the **Leaflet library** serves as the core JavaScript engine for rendering and managing the map visualization within the application.

Its primary roles include:

- **Runtime Map Injection:** It is used as a JavaScript OpenStreetMap (OSM) library that is injected at runtime into a native application via the Microsoft Edge WebView2 component.
- **Interactive Visualization:** The library is part of an HTML/JS template (referenced as `HOVER_JS_SCRIPT2`) that displays map coordinates and provides features like “Hover Coordinates” on the OpenStreetMap interface.
- **Dynamic Data Rendering:** It enables the application to take specific user inputs — such as **latitude, longitude, and zoom levels** — and translate them into a live, updated map view.
- **Web-to-Native Integration:** It allows the developer to embed complex web content (HTML, CSS, and JS) into a native Delphi or maXbox rich client, leveraging the EdgeView2 runtime to display geographic data without needing a standalone web browser.

Mastering WebView Injection: Dynamic Mapping with EdgeView2

Phase 1: Data Capture and Formatting



Latitude:

Longitude:

Zoom:

Desktop native application



Capturing Coordinate Input

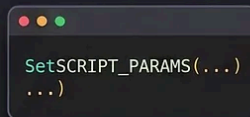
Extracts Latitude, Longitude, and Zoom values from the application's TEdit UI controls.

```
<script rconurcation('Launade', SetSCRIPT_PARAMS(..) >
```



Parameter Formatting

Converts text to floats and formats them to precise decimal places for script injection.



```
SetSCRIPT_PARAMS(...)
```

Parameter Validation

Calls the **SetSCRIPT_PARAMS** function to initialize the injection logic within the component.

Phase 2: Script Processing and Display

```
1 functions Edget()
2 try {
3   coordinas.logior(
4     HOVER_JS_SCRIPT2 {
5       lat..ned("lat, lon!"):
6       lon. new L.earker([lat, lon])
7     };
8   };
9   console.ixlogic 'edited';
10 };
11 end
```

Template Script Processing

Updates the **HOVER_JS_SCRIPT2** template with new coordinates while handling UTF-8 encoding.

```
<script>
copy..
...nev = new L.marker([lat, lon])...
...new L.marker([lat, lon])...
</script>
```

UI Component Integration

Pushes the processed HTML/JS code into a memo control for the browser viewer.

Phase 3: Programmatic View Refresh



Microsoft Edge
WebView2



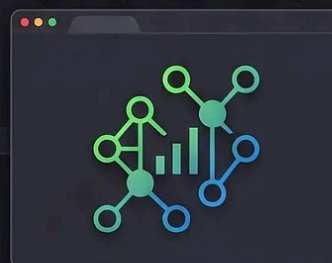
Programmatic View Refresh

Triggers a refresh event to update the Microsoft Edge WebView2 OpenStreetMap display.

Phase 4: Final Output



Leaflet/
OpenStreetMap



Dynamic Mapping

The updated script is executed, dynamically refreshing the map visualization with the new coordinates and parameters.

NotebookLM

Summary and Conclusion Prompting

A second solution gets the JavaScript from a host and will be injected with stringreplace direct after the download on load:

Be aware: to find `L.map("map").setView` in JS you have to use double quotes, otherway stringreplace function can't find it! (see below)

```
procedure TEdit1Button1Click(Sender: TObject);
var htmlgeojs2: string;
begin
  writ('debc: '+edt1lat.text+' '+edt2long.text)
  htmlgeojs2:= utf8decode(HttpGetDirect2(GEOJSON4));
  htmlgeojs2:= (stringreplace((htmlgeojs2),
    'const map = L.map(''map'').setView([46.948, 7.44
    'const map = L.map("map").setView(['+(edt1lat.tex
      +(edt2long.text)+'],'+flots(zoomf)+'));
    [rfReplaceAll]));
  htmlgeojs2:= (stringreplace((htmlgeojs2),
    '"coordinates": [7.447422, 46.947945]',
    '"coordinates": ['+(edt2long.text)+'+',+(edt1lat.t
      [rfReplaceAll]));    //}
  //htmlgeojs2:= utf8encode(stringreplace((HOVER_JS_SCRIPT2),LF,CRLF,[rfRepl
  htmlgeojs2:= utf8encode(stringreplace((htmlgeojs2),LF,CRLF,[rfReplaceAll]))
  edgeviewfrm.memoHTML.Text:= utf8toansi(htmlgeojs2); ///GEOJSON_Layer;
  edgeviewfrm.btnSetSourceClick(self);
end;
```



back on track from Oslo to Hamburg

ウェブビューの挿入

Geography +

Geospatial +

Geocoding +

Geocoding Map +